

Design First

OpenApi et AsyncApi en action !



Philippe Bousquet



Qui suis-je ?



Philippe Bousquet <pbousquet@sqli.com>

Architecte / Directeur Technique chez SQLI



[philippe bousquet](#)



[darken33](#)



Archilocus



Bordeaux
JUG



sqli

#S14

1990

Création

2200

Employés

6

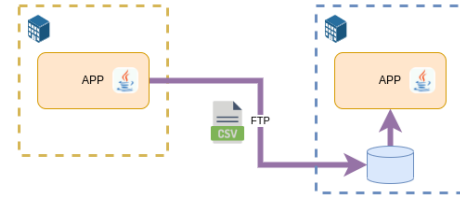
Agences en France



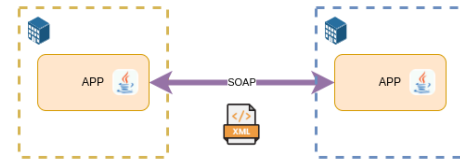
Un peu d'histoire

Un peu d'histoire

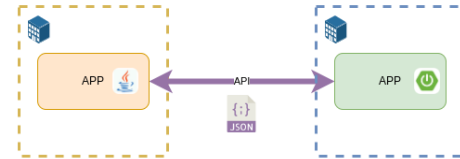
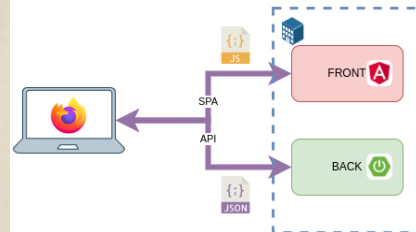
2000's



2005/2010's



2020's



Une démarche API



Une démarche API



Normalisation

- Définition des standards du SI et normalisation



Architecture

- Approche Domain Driven Design et Architecture Hexagonale



Spécification

- Approche Design First pour les spécifications et la documentation



Implémentation

- Se baser sur de bonnes pratiques de développement



Testabilité

- Garantir la qualité au travers de la pyramide de tests



Management

- Mise en place d'outils de gouvernance des API



Observabilité

- Mise en place d'outils d'observabilité et de monitoring

Une démarche API



Normalisation

- Définition des standards du SI et normalisation



Architecture

- Approche Domain Driven Design et Architecture Hexagonale



Specification

- Approche Design First pour les spécifications et la documentation



Implementation

- Se baser sur de bonnes pratiques de développement



Testabilité

- Garantir la qualité au travers de la pyramide de tests



Management

- Mise en place d'outils de gouvernance des API



Observabilité

- Mise en place d'outils d'observabilité et de monitoring

Code First

Code First

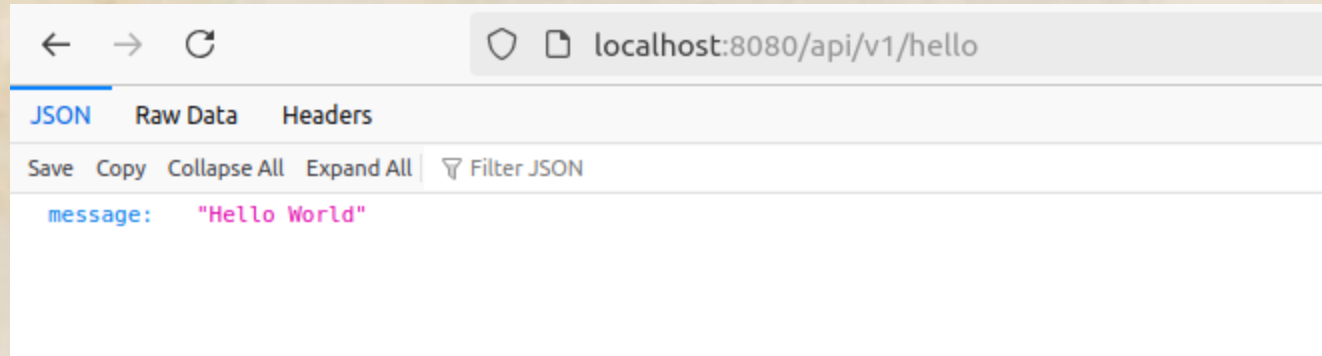
On ouvre son IDE et on code son API

```
@RestController
public class HelloApi {

    @RequestMapping(
        method = RequestMethod.GET,
        value = "/api/v1/hello",
        produces = { "application/json" }
    )
    public ResponseEntity<HelloDto> hello() {
        HelloDto result = new HelloDto();
        result.setMessage("Hello World");
        return ResponseEntity.ok(result);
    }
}
```

Code First

Et cela fonctionne, on ouvre le navigateur



Aucune documentation de notre API

Code First

On peut cependant ajouter des annotations OpenApi

```
@RequestMapping(  
    ...  
)  
@Operation(  
    operationId = "helloWithName",  
    summary = "Saluer une personne en particulier",  
    tags = { "Hello" },  
    responses = {  
        @ApiResponse(  
            responseCode = "200",  
            description = "OK",  
            content = {  
                @Content(  
                    mediaType = "application/json",  
                    schema = @Schema(implementation = HelloDto.class)  
                )  
            }  
        )  
    },  
)  
public ResponseEntity<HelloDto> hello() {  
    ...  
}
```


Code First

Pour résumer :

- Coder l'API directement dans son IDE
- Documentation inexistante
- L'API doit être disponible pour développer les consommateurs
- Des outils existent pour générer la Documentation

Design First





Design First

- Spécifier son API en amont
- Implémenter plus efficacement son API
- Faciliter l'intégration par les consommateurs
- Simuler le fonctionnement de l'API
- Avoir une documentation en adéquation avec le code

OpenApi & AsyncApi



STANDARD

OpenApi & AsyncApi

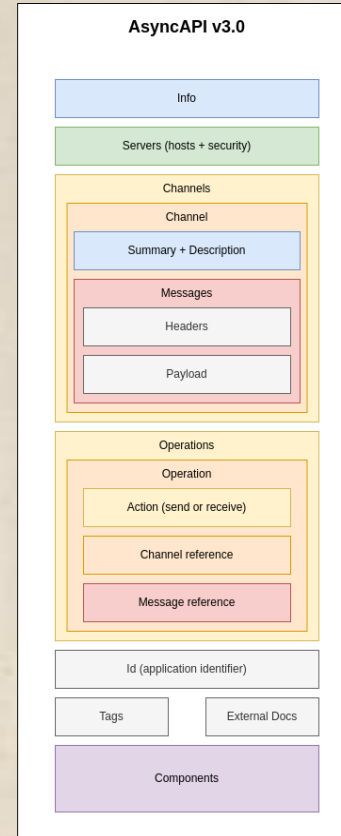
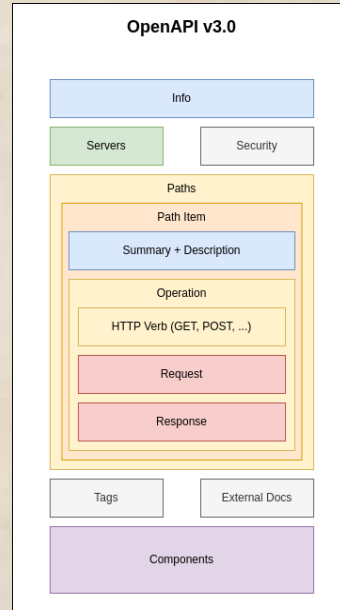
Année			
2011	v1.0		
2014	v2.0		
2015	Smart Bear cède Swagger		
2017		v3.0.0	Fran Méndez
2019			v2.0.0
2021		v3.1.0	v2.2.0
2022			v2.5.0
2023			v3.0.0
2025		v3.2.0	

STANDARD

Les initiatives

		
Site web	https://www.openapis.org/	https://www.asyncapi.com/
Linux Foundation	✓	✓
Spécifications	✓	✓
Documentation	✓	✓
Outillage	✗	✓
Communauté	✓	✓

Les spécifications

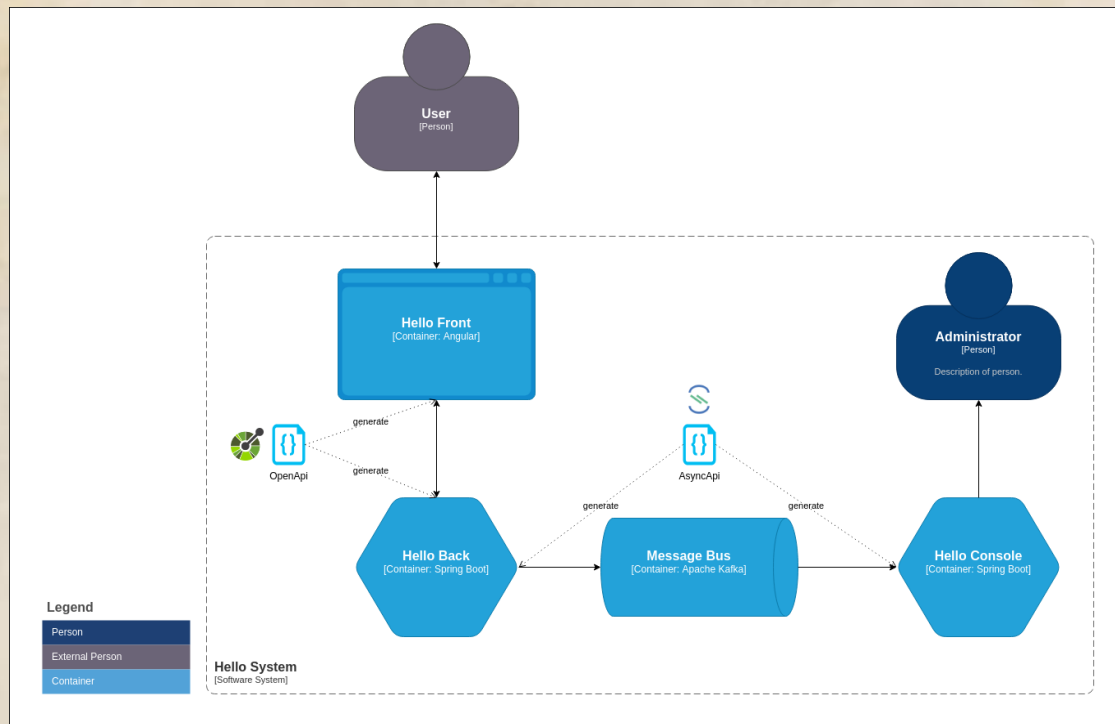




OpenApi : notre cas d'usage

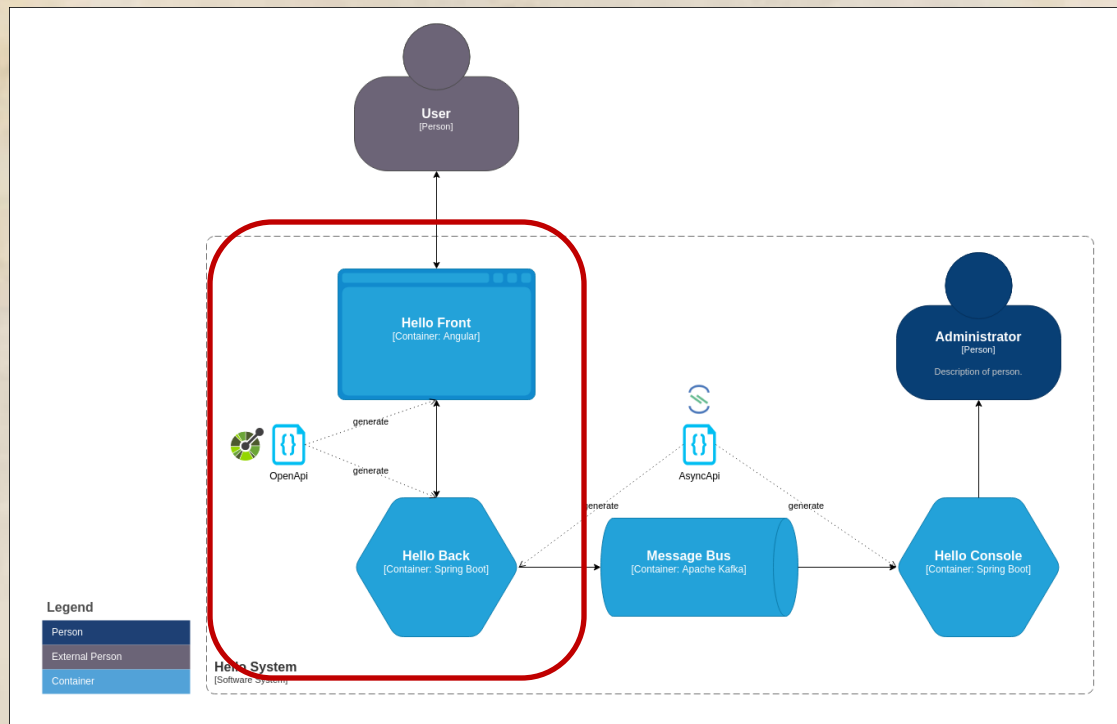


OpenApi : notre cas d'usage





OpenApi : notre cas d'usage





OpenApi : Designer

OpenApi : Designer



Swagger Editor

```
1 openapi: 3.0.1
2 info:
3   title: Connaissance Client API
4   description: Documentation de l'API Connaissance Client
5   version: "1.0.0"
6   contact:
7     name: SQL
8     url: http://sql.com/
9     email: pousquet@sql.com
10  license:
11    name: Copyright (c) SQL
12  servers:
13    - url: http://localhost:8080/
14      description: DEV Server
15      variables: {}
16    - url: https://jenesaispas/
17      description: Azure Cloud
18      variables: {}
19  tags:
20    - name: ConnaissanceClient
21      description: opérations de lecture / écriture sur les fiches de connaissance client
22  security:
23    - bearerAuth: []
24  paths:
25    /v1/connaissance-clients:
26      get:
27        security: []
28        tags:
29          - ConnaissanceClient
30        summary: Liste des Fiches de connaissance client
31        operationId: getConnaissanceClients
32        description: Liste des fiches de connaissance client
33        responses:
34          200:
35            description: Réponse pour une requête valide
36            content:
37              application/json:
38                schema:
39                  $ref: '#/components/schemas/ConnaissanceClients'
40          400:
41            description: Invalid Request
42            content:
43              application/json:
44                schema:
45                  $ref: '#/components/schemas/ApiErrorResponse'
46          401:
47            description: Access forbidden
48            content:
49              application/json:
50                schema:
51                  $ref: '#/components/schemas/ApiErrorResponse'
52          403:
53            description: Access denied
54            content:
55              application/json:
56                schema:
57                  $ref: '#/components/schemas/ApiErrorResponse'
58          409:
59            description: Conflict
60            content:
61              application/json:
62                schema:
63                  $ref: '#/components/schemas/ApiErrorResponse'
64        post:
65          tags:
66            - ConnaissanceClient
67          summary: Accès en écriture à une fiche de connaissance client
```

Try our new Editor ↗

Connaissance Client API ^{1.0.0} ^{OAS 3.0}

<https://raw.githubusercontent.com/openmeterio/openmeter/main/api/openapi.yaml>

Documentation de l'API Connaissance Client

[SQL - Website](#)
[Send email to SQL](#)
[Copyright \(c\) SQL](#)

Servers

[Authorize](#)

ConnaissanceClient opérations de lecture / écriture sur les fiches de connaissance client

- GET** /v1/connaissance-clients Liste des fiches de connaissance client
- POST** /v1/connaissance-clients Accès en écriture à une fiche de connaissance client
- GET** /v1/connaissance-clients/{id} Accès en lecture à une fiche de connaissance client
- DELETE** /v1/connaissance-clients/{id} Suppression d'une fiche de connaissance client
- PUT** /v1/connaissance-clients/{id}/adresse Changement d'adresse du client
- PUT** /v1/connaissance-clients/{id}/situation Changement de situation du client

Schemas

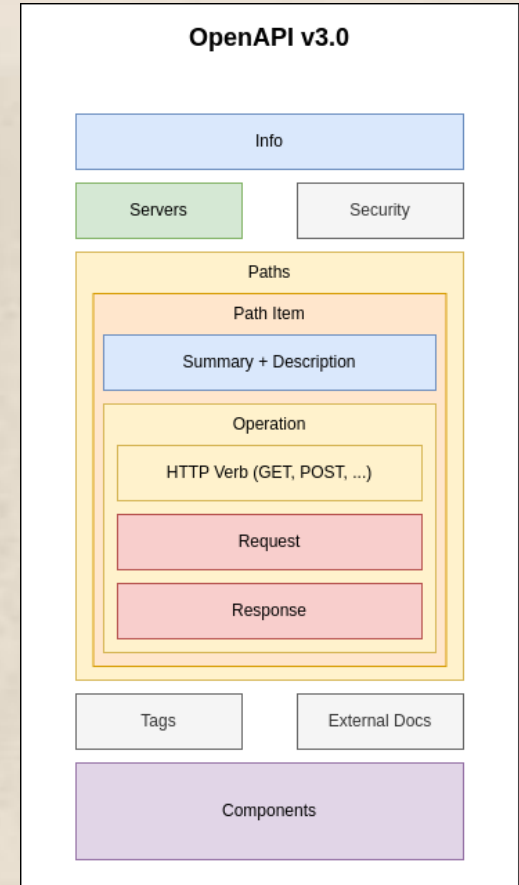
[ApiErrorResponse](#)

<https://editor.swagger.io/>



OpenApi : Designer

- Les routes
- Les verbes HTTP
- Les paramètres & réponses
- Les contrôles
- La sécurité
- La documentation



OpenApi : Tooling

Tool Types

We've organised everything into categories so you can jump to the section you're interested in.

- **Auto Generators:** Tools that will take your code and turn it into an OpenAPI Specification document
- **Converters:** Various tools to convert to and from OpenAPI and other API description formats.
- **Data Validators:** Check to see if API requests and responses are lining up with the API description.
- **Description Validators:** Check your API description to see if it is valid OpenAPI.
- **Documentation:** Render API Description as HTML (or maybe a PDF) so slightly less technical people can figure out how to work with the API.
- **DSL:** Writing YAML by hand is no fun, and maybe you don't want a GUI, so use a Domain Specific Language to write OpenAPI in your language of choice.
- **Gateways:** API Gateways and related tools that have integrated support for OpenAPI.
- **GUI Editors:** Visual editors help you design APIs without needing to memorize the entire OpenAPI specification.
- **Learning:** Whether you're trying to get documentation for a third party API based on traffic, or are trying to switch to design-first at an organization with no OpenAPI at all, learning can help you move your API spec forward and keep it up to date.
- **Miscellaneous:** Anything else that does stuff with OpenAPI but hasn't quite got enough to warrant its own category.
- **Mock Servers:** Fake servers that take description document as input, then route incoming HTTP requests to example responses or dynamically generates examples.
- **Monitoring:** Monitoring tools let you know what is going on in your API.
- **Parsers:** Loads and read OpenAPI descriptions, so you can work with them programmatically.
- **SDK Generators:** Generate code to give to consumers, to help them avoid interacting at a HTTP level.
- **Security:** By poking around your OpenAPI description, some tools can look out for attack vectors you might not have noticed.
- **Server Implementations:** Easily create and implement resources and routes for your APIs.
- **Testing:** Quickly execute API requests and validate responses on the fly through command line or GUI interfaces.
- **Text Editors:** Text editors give you visual feedback whilst you write OpenAPI, so you can see what docs might look like.

<https://openapi.tools/>



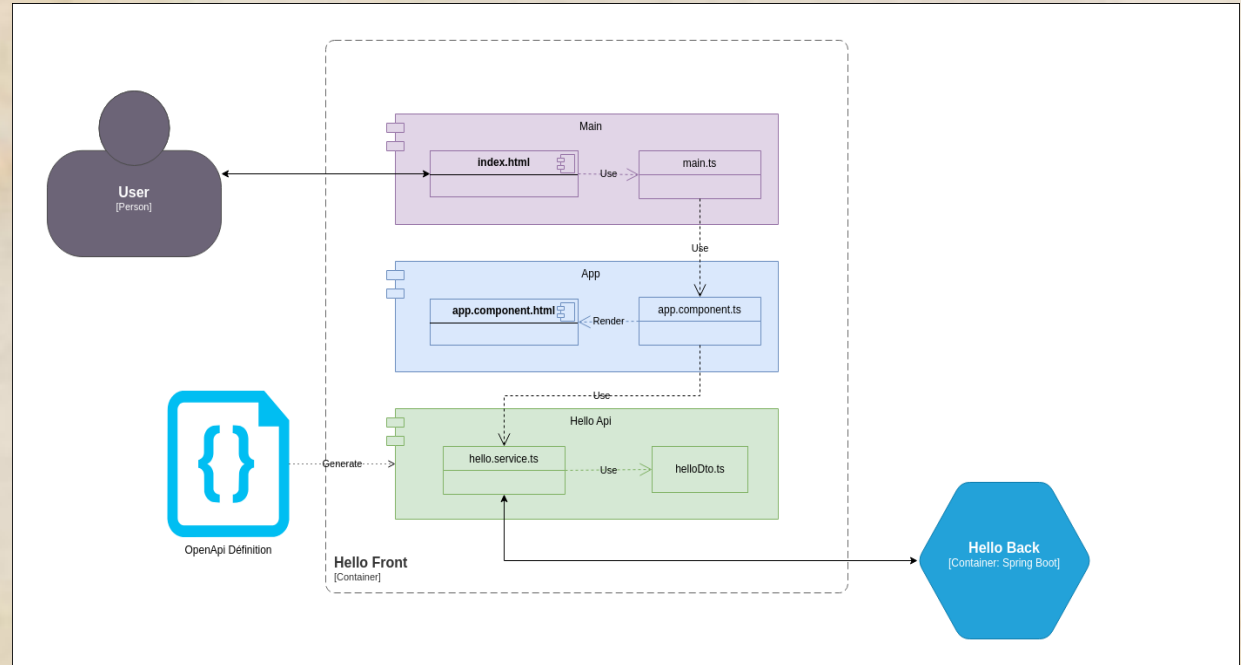
Démo

Génération client



Génération client

openapi-generator : DTOs, client HTTP, pour plusieurs langages





Génération client

Il suffit alors d'intégrer l'appel dans son code

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { HelloService } from '../hello-api/api/hello.service';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  title = 'Appel de l\'api hello';
  constructor(private helloService: HelloService) {
    this.helloService.helloWorld().subscribe(helloDto => {
      this.title = helloDto.message!;
    });
  }
}
```

<http://localhost:4200/>



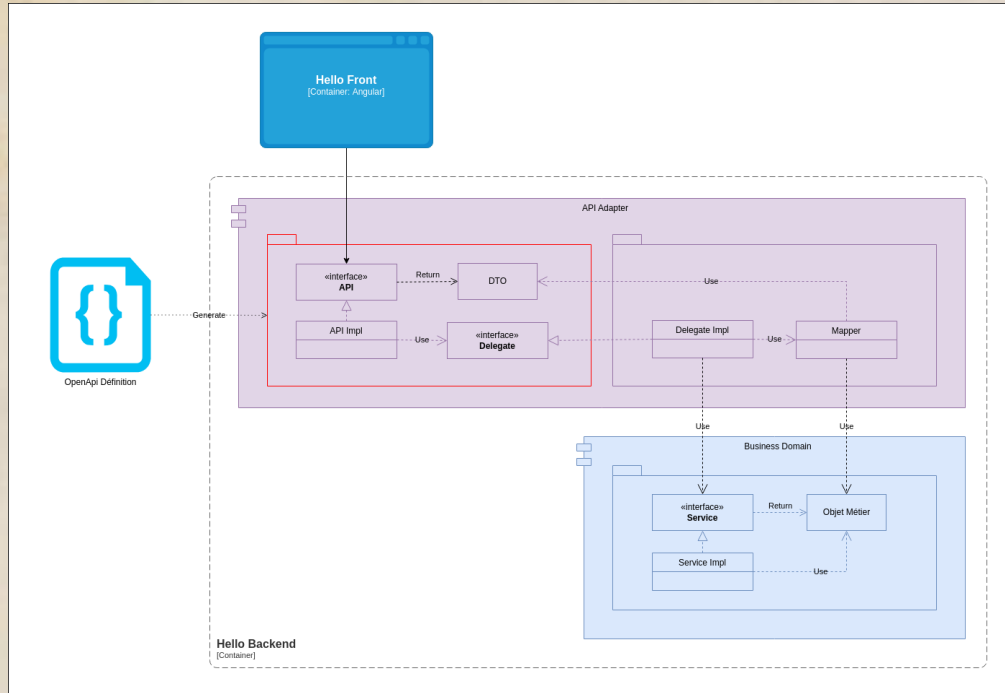
Démo

Génération server



Génération server

openapi-generator : DTOs, exposition API, pour plusieurs langages



Génération server

Il suffit alors d'implémenter l'interface **delegate** générée

```
@Component
public class HelloApiDelegateImpl implements HelloApiDelegate {

    private final HelloService helloService;

    public HelloApiDelegateImpl(HelloService helloService) {
        this.helloService = helloService;
    }

    @Override
    public ResponseEntity<HelloDto> helloWorld() {
        HelloDto result = new HelloDto();
        result.setMessage(helloService.sayHello("World"));
        return ResponseEntity.ok(result);
    }
    ...
}
```

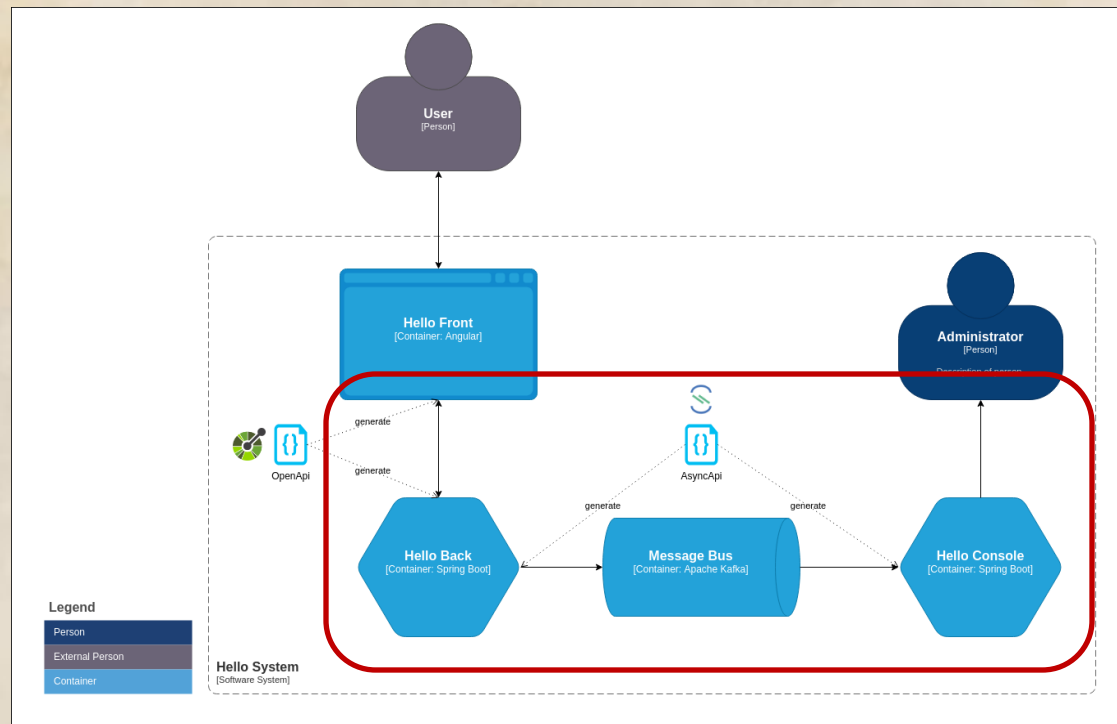
<http://localhost:8080/swagger-ui/index.html>



Démo



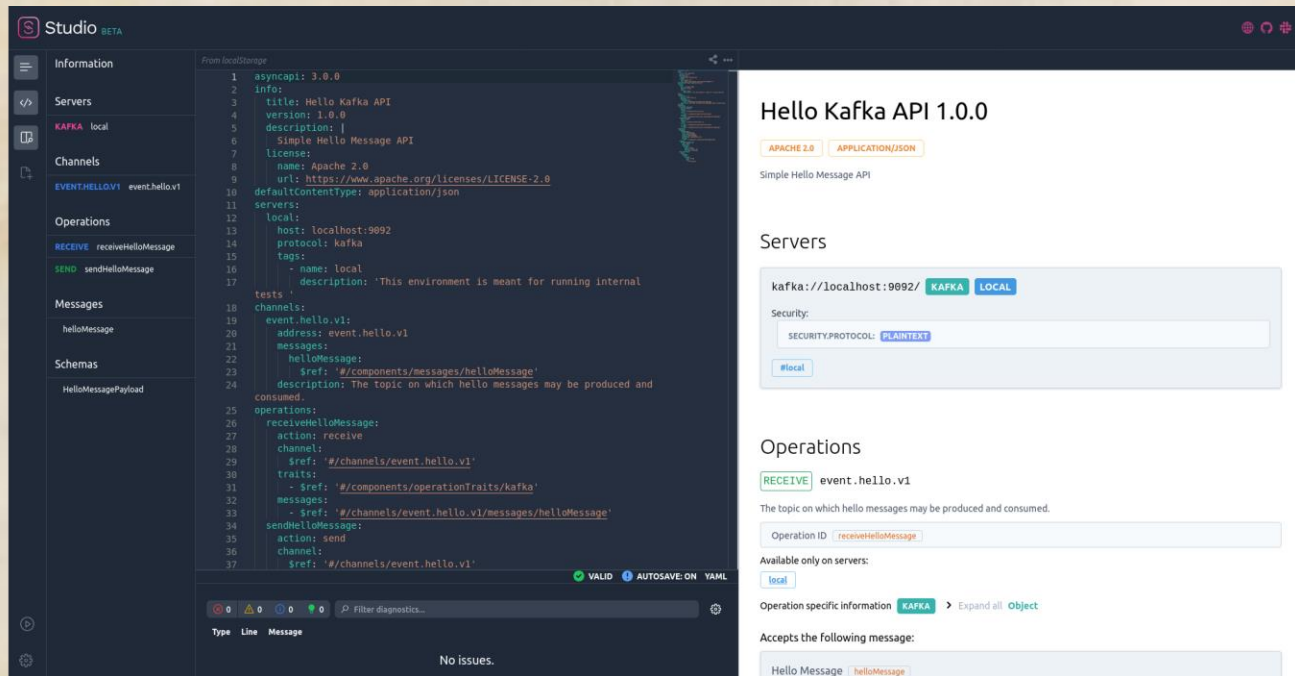
AsyncApi : notre cas d'usage





AsyncApi : Définir l'API

AsyncApi : Définir l'API



The screenshot displays the AsyncAPI Studio interface, which is used for defining and testing AsyncAPI specifications. The central editor shows a YAML definition for a service named 'Hello Kafka API 1.0.0'. The definition includes metadata (title, version, description, license, url), server information (localhost:9092), channels (event.hello.v1), and operations (receiveHelloMessage and sendHelloMessage). The interface also features a left sidebar with navigation options like Information, Servers, Channels, Operations, Messages, and Schemas. On the right, a preview pane shows the rendered API documentation, including the title, description, servers, and operations sections.

```
1 asyncapi: 3.0.0
2 info:
3   title: Hello Kafka API
4   version: 1.0.0
5   description: |
6     Simple Hello Message API
7   license:
8     name: Apache 2.0
9     url: https://www.apache.org/licenses/LICENSE-2.0
10  defaultContentType: application/json
11  servers:
12    local:
13      host: localhost:9092
14      protocol: kafka
15      tags:
16        - name: local
17          description: 'This environment is meant for running internal tests'
18  channels:
19    event.hello.v1:
20      address: event.hello.v1
21      messages:
22        helloMessage:
23          $ref: '#/components/messages/helloMessage'
24      description: The topic on which hello messages may be produced and consumed.
25  operations:
26    receiveHelloMessage:
27      action: receive
28      channel:
29        $ref: '#/channels/event.hello.v1'
30      traits:
31        - $ref: '#/components/operationTraits/kafka'
32      messages:
33        - $ref: '#/channels/event.hello.v1/messages/helloMessage'
34    sendHelloMessage:
35      action: send
36      channel:
37        $ref: '#/channels/event.hello.v1'
```

Hello Kafka API 1.0.0

Simple Hello Message API

Servers

kafka://localhost:9092/ **KAFKA** **LOCAL**

Security:

SECURITY.PROTOCOL: **PLAINTEXT**

#local

Operations

RECEIVE event.hello.v1

The topic on which hello messages may be produced and consumed.

Operation ID: **receiveHelloMessage**

Available only on servers:

local

Operation specific information **KAFKA** > Expand all Object

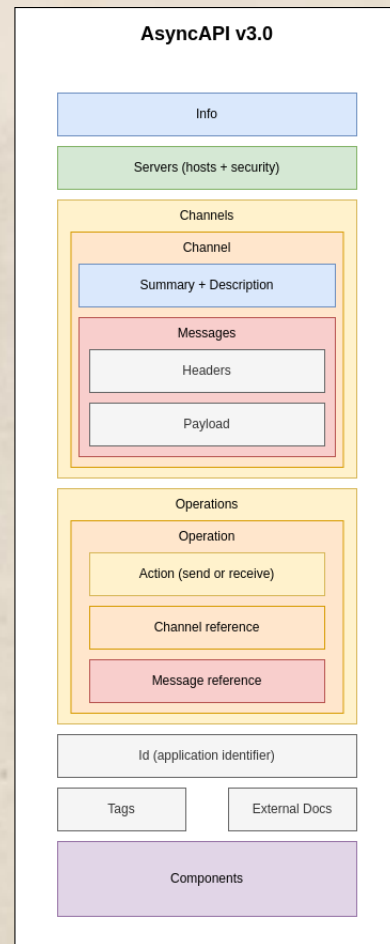
Accepts the following message:

Hello Message: **helloMessage**

<https://studio.asyncapi.com/>

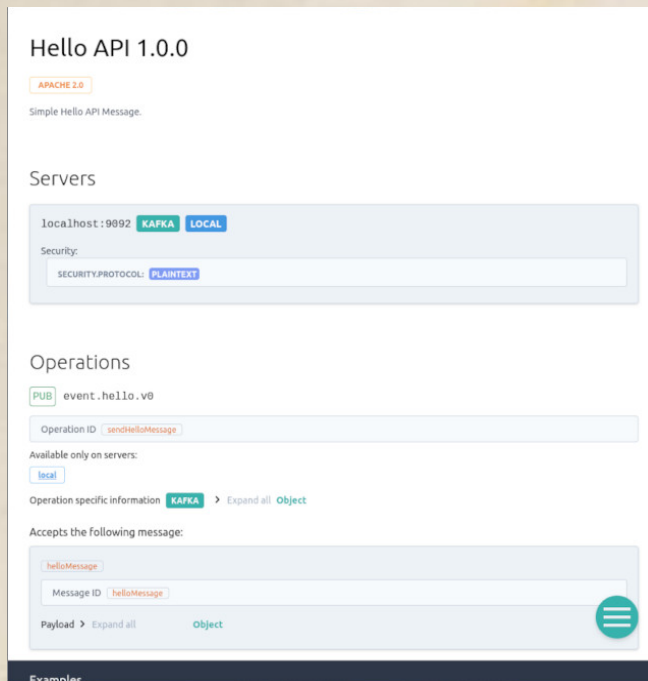
AsyncApi : Designer

- Le serveur + sécurité
- Le chanel
- Le message
- Les opérations (send, receive)
- La documentation



AsyncApi : Tools

- HTML Template
- React Template
- Spring Template
- NodeJS Template
- PHP Template
- Et divers autres...



<https://www.asyncapi.com/tools>



Démo

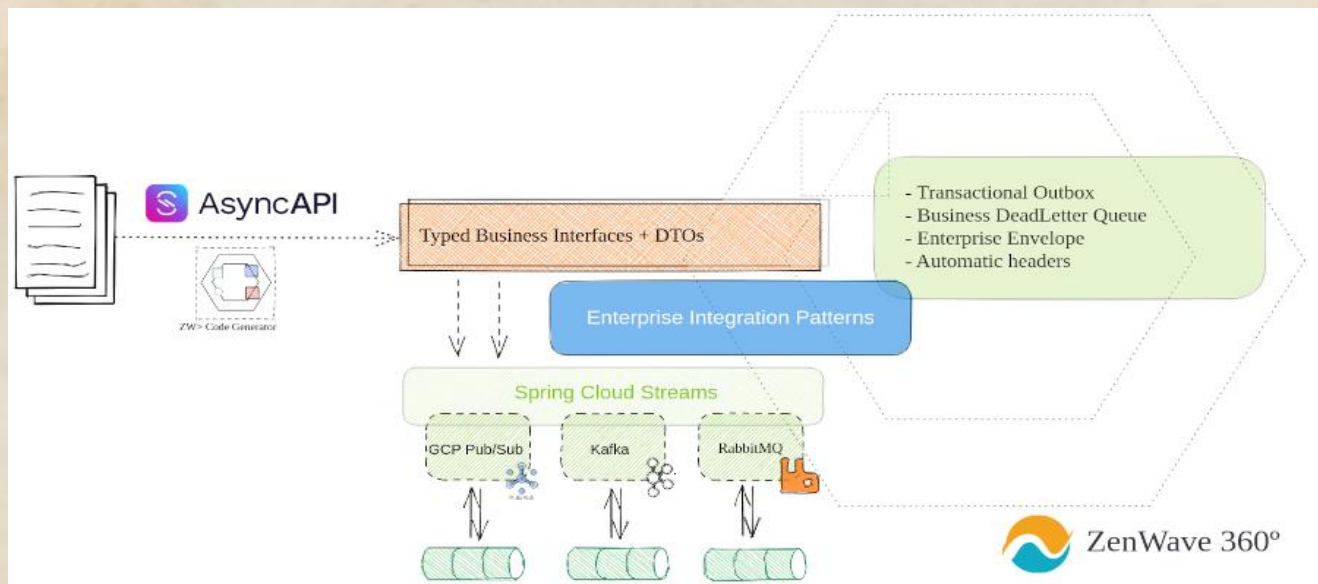
ZenWave SDK





ZenWave SDK

Accélérer le développement d'**API First**



<https://www.zenwave360.io/zenwave-sdk/>

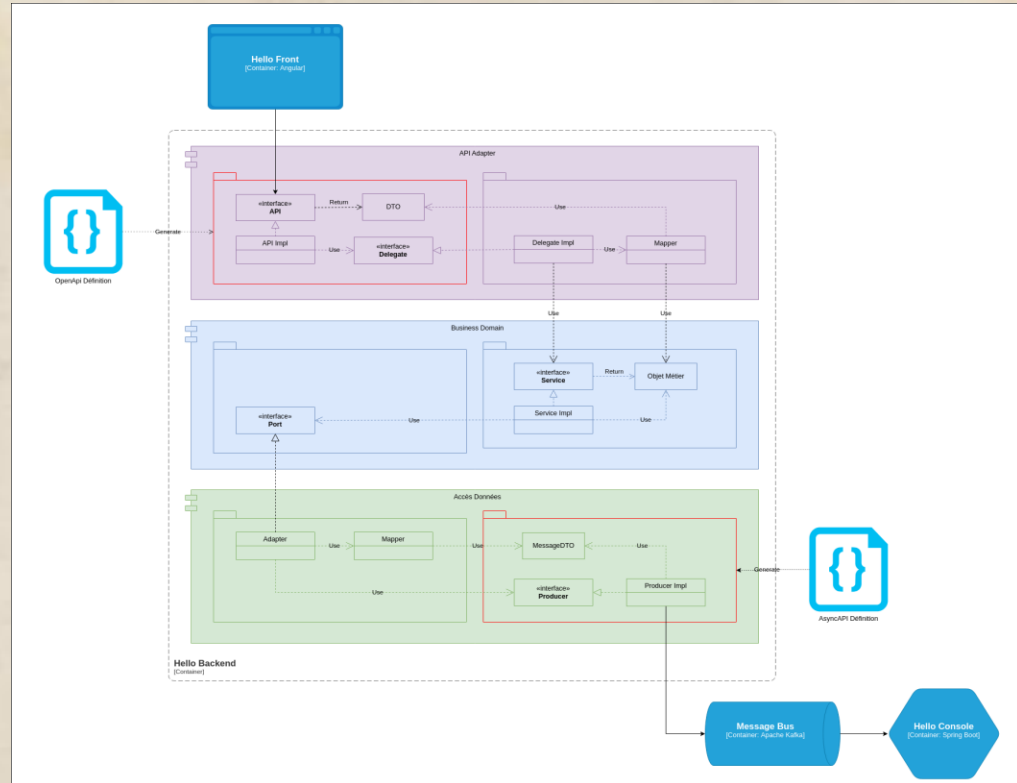


Génération du Producer





Génération du Producer





Génération du Producer

Le plugin Maven propose plusieurs générateurs dont :

jsonschema2pojo : pour générer les DTOs

- Définir **<modelPackage>** : package java pour le code
- Définir **<isUseJakartaValidation>** : pour java 17 / spring-boot 3

spring-cloud-streams3 : pour générer le code basé sur spring-cloud-streams

- Définir **<apiPackage>** : package java pour le code généré
- Définir **<modelPackage>** : package java pour le code
- Définir **<opérationIds>** : les opérations à générer (send)



Génération du Producer

Il suffit alors d'implémenter le service

```
@Service
public class HelloServiceImpl implements HelloService{

    private final DefaultServiceEventsProducer service;

    public HelloServiceImpl(DefaultServiceEventsProducer service) {
        this.service = service;
    }

    @Override
    public String sayHello(String name) {
        String message = "Hello " + name;
        HelloMessagePayload payload = new HelloMessagePayload();
        payload.setMessage(message);
        service.sendHelloMessage(payload);
        return message;
    }
}
```

<http://localhost:8080/swagger-ui/index.html>

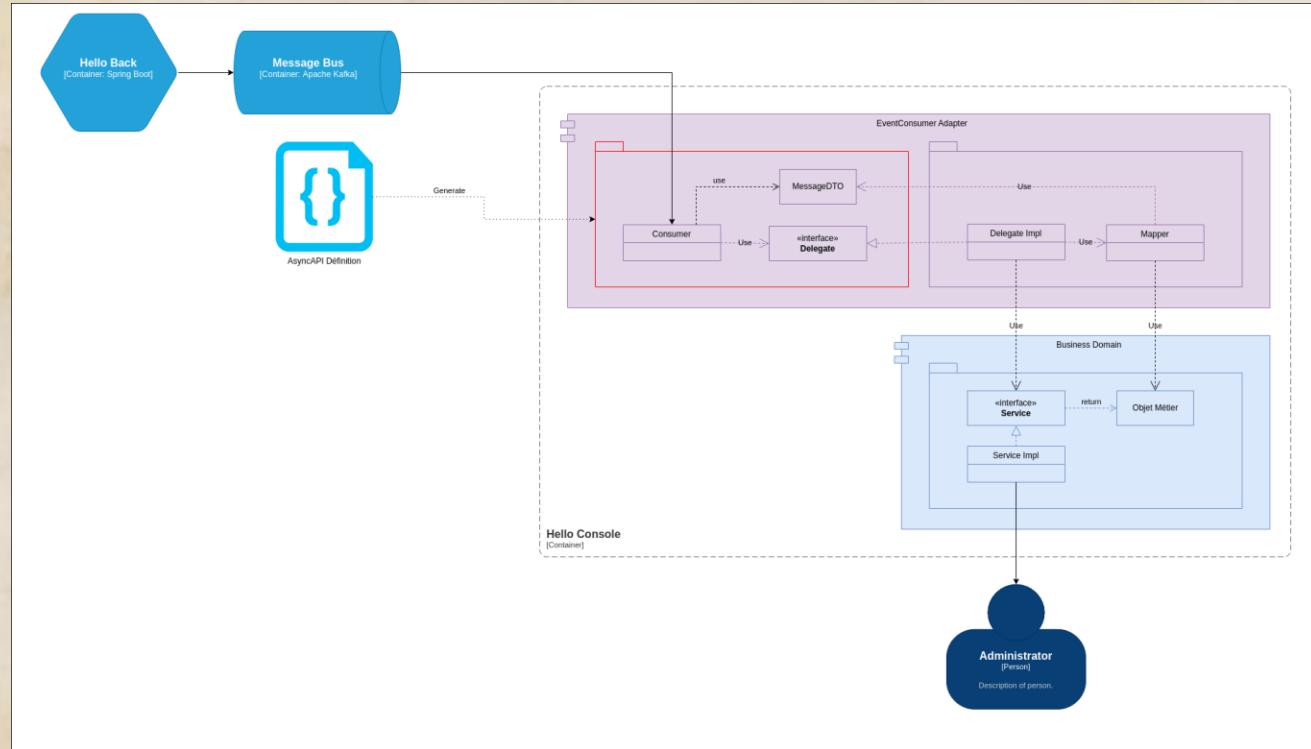


Démo

Génération consumer



Génération consumer





Génération consumer

Le plugin Maven propose plusieurs générateurs dont :

jsonschema2pojo : pour générer les DTOs

- Définir **<modelPackage>** : package java pour le code
- Définir **<isUseJakartaValidation>** : pour java 17 / spring-boot 3

spring-cloud-streams3 : pour générer le code basé sur spring-cloud-streams

- Définir **<apiPackage>** : package java pour le code généré
- Définir **<modelPackage>** : package java pour le code
- Définir **<opérationIds>** : les opérations à générer (receive)



Génération consumer

Il suffit alors d'implémenter le service

```
@Component
public class HelloService implements
    IReceiveHelloMessageConsumerService {

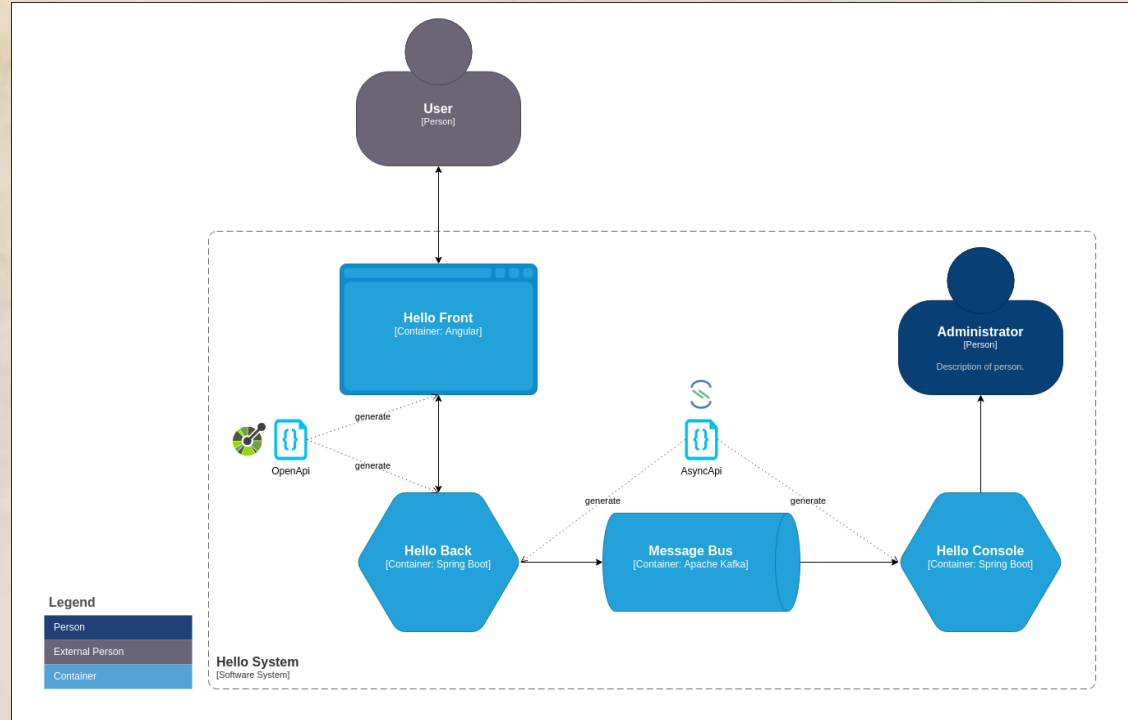
    @Override
    public void receiveHelloMessage(
        HelloMessagePayload payload,
        HelloMessagePayloadHeaders headers) {
        System.out.println(payload.getMessage());
    }
}
```



Démo



Notre cas d'usage





Pour conclure

Pour conclure

- OpenApi & AsyncApi
- Spécifications + Outillage
- Doit permettre :
 - Implémenter plus efficacement vos apis
 - Intégrer plus facilement les appels à vos apis
 - Possibilité de simuler vos apis
 - Documentation en adéquation avec l'implémentation réelle



Merci

La présentation



Vos Feedbacks



Thank you



Philippe Bousquet

